

双路高速 DA 实验

DAC (Digital to Analog Converter, 即数模转换器) 是大多数系统中必不可少的组成部件, 用于将离散的数字信号转换成连续的模拟信号, 它们是连接模拟电路和数字电路必不可少的桥梁。在很多场合下, DAC 的转换速度甚至直接决定了整个系统的运行速度。本章我们将使用双路高速 DA 模块实现数模转换, 并产生两路正弦波模拟电压信号。

本章包括以下几个部分:

- 1.1 简介
- 1.2 硬件设计
- 1.3 实验任务
- 1.4 程序设计
- 1.5 下载验证
- 1.6 本章总结
- 1.7 拓展训练

1.1 简介

本章我们使用的双路 DA 模块是正点原子推出的一款双路高速数模转换模块 (ATK_DUAL_HS_DA)，高速 DA 转换芯片是由思瑞浦公司生产的 3PD5651E 芯片。

ATK_DUAL_HS_DA 模块的硬件结构图如下图所示。

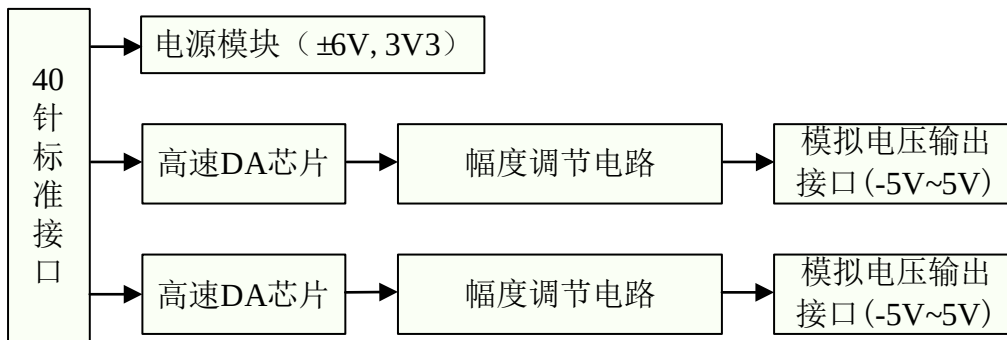


图 1.1.1 ATK_DUAL_HS_DA 模块硬件结构图

DA 芯片 (3PD5651E) 输出的是一对差分电流信号，通过高性能和高带宽的运放电路，实现差分变单端以及幅度调节等功能，使整个电路的性能得到了最大限度的提升，最终输出的模拟电压范围是 -5V~+5V。

下面来介绍下这款芯片。

3PD5651E 是 3PEAK 公司 (思瑞浦微电子科技股份有限公司) 生产的 DAC 系列数模转换器，具有高性能、低功耗的特点。3PD5651E 的数模转换位数为 10 位 (即分辨率为 $2^{10}=1024$)，最大转换速度为 125MSPS (每秒采样百万次，Million Samples per Second)。

3PD5651E 的内部功能框图如下图所示：

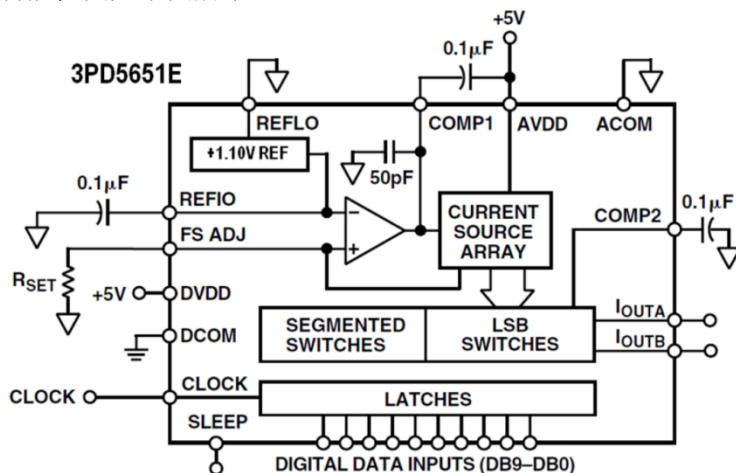


Figure 2. Functional Block Diagram

图 1.1.2 3PD5651E 的内部功能框图

3PD5651E 在时钟 (CLOCK) 的驱动下工作，内部集成了 +1.1V 参考电压 (+1.10V REF)、运算放大器、电流源 (CURRENT SOURCE ARRAY) 和锁存器 (LATCHES)。两个电流输出端 IOUTA 和 IOUTB 为一对差分电流，当输入数据为 0 (DB[9:0]=10'h000) 时，IOUTA 的输出电流为 0，而 IOUTB 的输出电流达到最大 (最大值的大小跟参考电压有关)；当输入数据全为高电平 (DB[9:0]=10'h3ff) 时，IOUTA 的输出电流达到最大 (最大值的大小跟参考电压有关)，而 IOUTB 的输出电流为 0。

3PD5651E 必须在时钟的驱动下才能把数据写入片内的锁存器中，其触发方式为上升沿触发，3PD5651E 的时序图如下图所示：

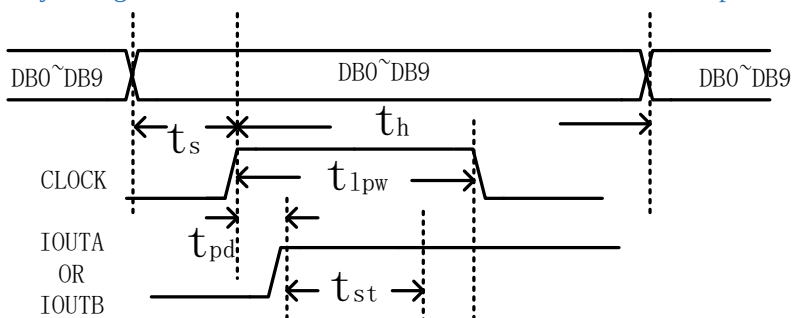


图 1.1.3 3PD5651E 时序图

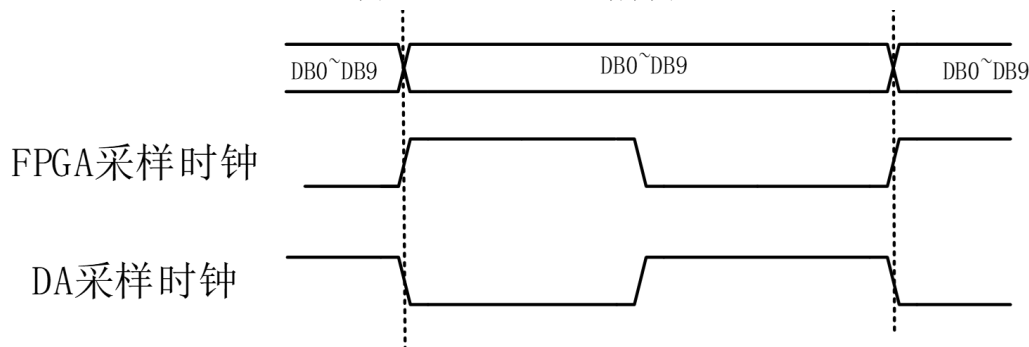


图 1.1.4 FPGA 内部时序

如图 1.1.3 中的 DB0~DB9 和 CLOCK 是 3PD5651E 的 10 位输入数据和输入时钟, IOUTA 和 IOUTB 为 3PD5651E 输出的电流信号。由图 1.1.3 可知, 数据在时钟的上升沿锁存, 因此我们可以在时钟的下降沿发送数据, 这样可以使 DA 芯片在数据的中央采样, 保证数据采样的准确性, 如图 1.1.4 所示。需要注意的是, CLOCK 的时钟频率越快, 3PD5651E 的数模转换速度越快, 3PD5651E 的时钟频率最快为 125Mhz。

IOUTA 和 IOUTB 为 3PD5651E 输出的一对差分电流信号, 通过运放电路输出模拟电压信号, 电压范围是 -5V 至 +5V 之间。当输入数据等于 0 时, 3PD5651E 输出的电压值为 5V; 当输入数据等于 10'h3ff 时, 3PD5651E 输出的电压值为 -5V。

3PD5651E 是一款数字信号转模拟信号的器件, 内部没有集成 DDS (Direct Digital Synthesizer, 直接数字式频率合成器) 的功能, 但是可以通过控制 3PD5651E 的输入数据, 使其模拟 DDS 的功能。例如, 我们使用 3PD5651E 输出一个正弦波模拟电压信号, 那么我们只需要将 3PD5651E 的输入数据按照正弦波的波形变化即可, 下图为 3PD5651E 的输入数据和输出电压值按照正弦波变化的波形图。

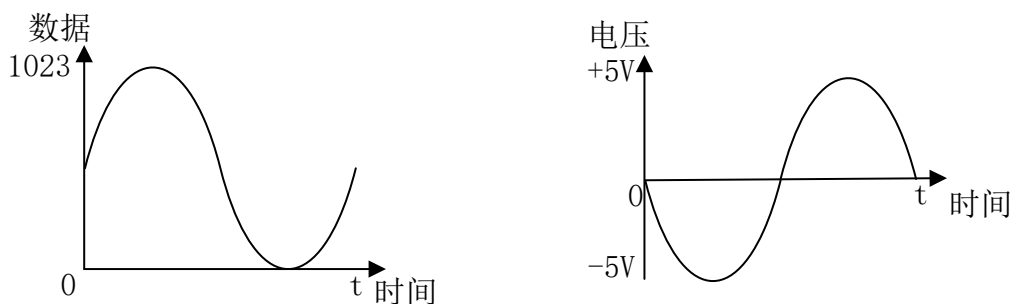


图 1.1.5 3PD5651E 正弦波数据 (左)、电压值 (右)

由上图可知, 数据在 0 至 1023 之间按照正弦波的波形变化时, 最终得到的电压也会按照正弦波的波形变化, 即输入数据如果循环按照正弦波的波形数据变化, 那么 3PD5651E 就可以持续不断的输出正弦波的模拟电压波形了。需要注意的是, 最终得到的 3PD5651E 的输出电压变化范围由其外部电路决定的, 当输入数据为 0 时, 3PD5651E 输出 +5V 的电压; 当输入数据为 1023 时, 3PD5651E 输出 -5V 的电压。

由此可以看出, 只要输入的数据控制的得当, 3PD5651E 可以输出任意波形的模拟电压信号, 例如正弦波、方波、锯齿波、三角波等波形。

1.2 硬件设计

ATK_DUAL_HS_DA 模块的实物图如下图所示:

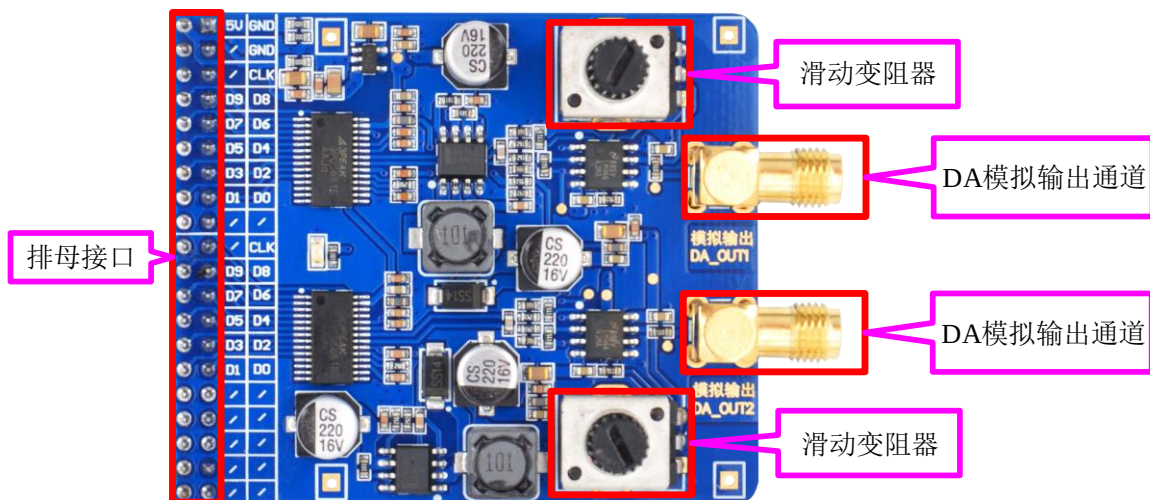


图 1.2.1 ATK_DUAL_HS_DA 模块实物图

ATK_DUAL_HS_DA 模块由 2 个型号为 3PD5651E 的 DA 转换芯片组成。3PD5651E 的原理图如下图所示:

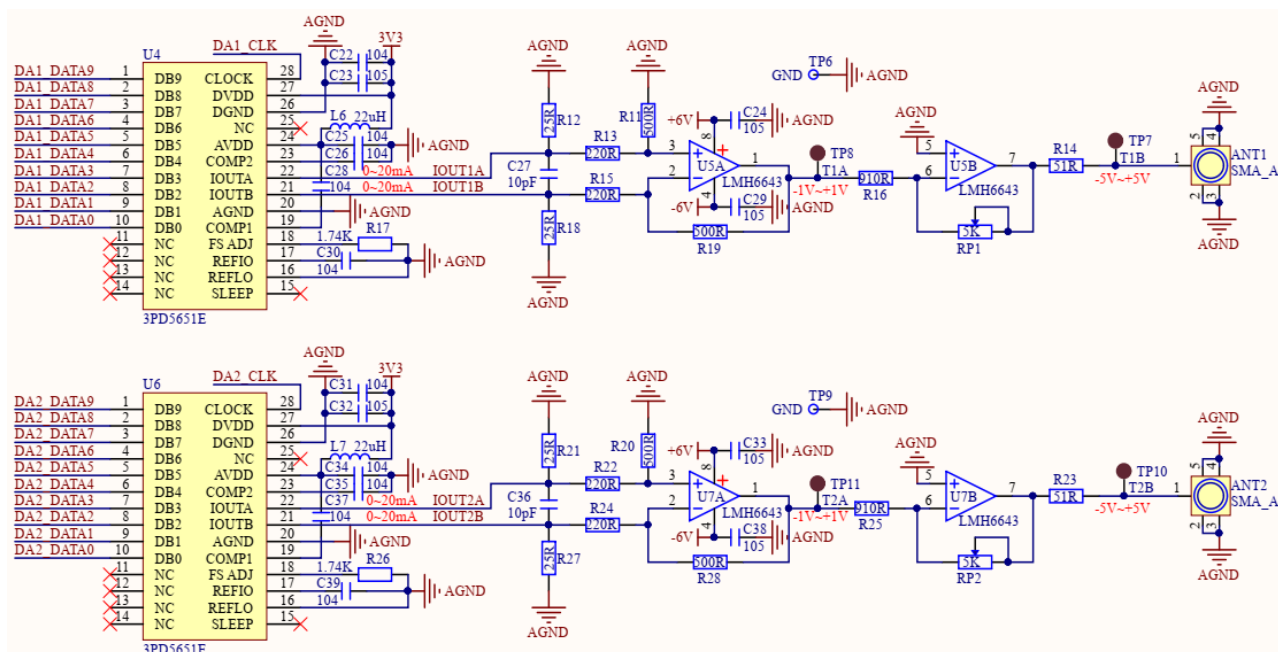


图 1.2.2 3PD5651E 原理图

由上图可知, 3PD5651E 输出的一对差分电流信号经过运放电路后得到了一个单端的模拟电压信号。图中右侧的 RP1 和 RP2 为滑动变阻器, 可以调节两路 DA 输出的电压范围, 推荐通过调节滑动变阻器, 使输出的电压范围在 -5V 至 +5V 之间, 从而达到 DA 转换芯片的最大转换范围。

1.3 实验任务

本章实验的任务是使用开发板及双路高速 DA 模块（ATK_DUAL_HS_DA 模块）实现数模转换。首先利用 FPGA 产生正弦波变化的数字信号，然后经过两片 DA 芯片转换成两路模拟信号（两路信号的波形和频率相同），最后通过示波器观察模拟信号的波形是否按照正弦波波形变化。

1.4 程序设计

1.4.1 总体模块设计

根据实验任务可知，本次设计使用双路高速 DA 模块，输出两路正弦波模拟信号，所以在双路 DA 模块进行数模转换之前，我们需要通过 FPGA 输出两路有效的波形数据。由于两路信号的波形和频率相同，所以 FPGA 只需要一个 DA 波形数据发送模块（da_wave_send）即可。波形数据发送模块所需的波形数据是通过 ROM 产生的，所以还需要一个 ROM IP 核。

此外，为了避免时钟抖动带来的波形毛刺，本次实验中用到时钟需要通过 PLL IP 核来进行输出，即我们使用 PLL IP 核来输出一路时钟，作为 DA 芯片的驱动时钟，同时该时钟也作为 ROM IP 核的读时钟。

至此，整个设计架构基本完善，综上所述，我们可以绘制出如下所示的系统架构图：

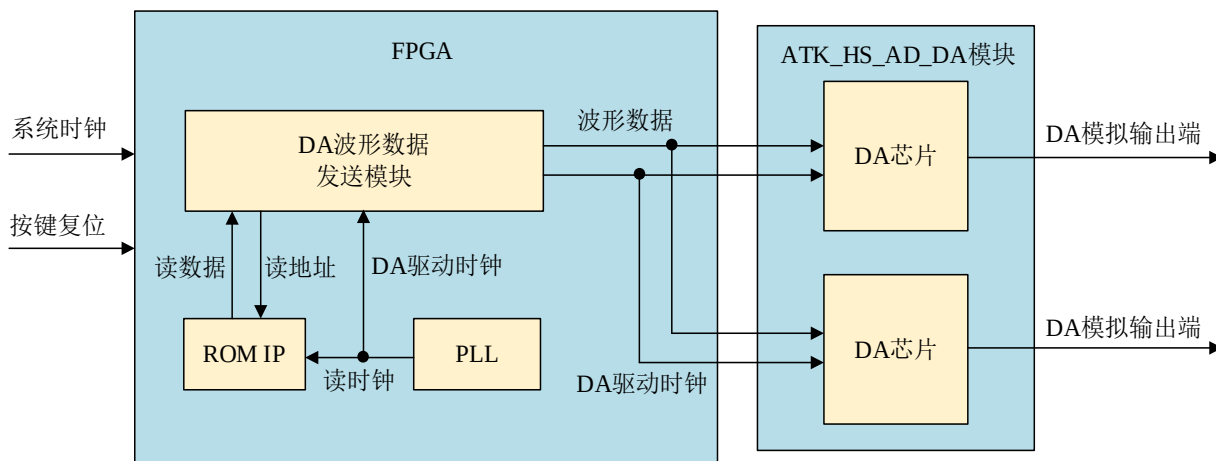


图 1.4.1 双路 DA 实验的系统架构图

接下来，我们分别对上图中的各个模块的功能做进一步的说明。

DA 波形数据发送模块（da_wave_send）：从上图可以看出，本实验中波形数据发送模块和“高速 AD/DA 实验”中的基本一致，虽然 FPGA 需要输出两路 DA 芯片驱动时钟（da_clk_1 和 da_clk_2）和两路波形数据（da_data_1 和 da_data_2），不过两路信号的波形和频率相同，所以只需要将波形数据发送模块的驱动时钟（da_clk）和波形数据（da_data）分别和两路 DA 芯片的驱动时钟信号和波形数据信号相连即可。

ROM IP 核（波形数据生成模块）：和“高速 AD/DA 实验”中一样，ROM IP 核需要读时钟（clk_50m）、读地址（rd_addr）和读数据（rd_data）三个信号端口。不同的是，由于本实验使用的双路 DA 模块的 DA 芯片数字信号位宽为 10bit，所以 ROM IP 核数据宽度也需要设置为 10bit。

PLL IP 核（时钟模块）：PLL IP 核使用 50MHz 系统时钟（sys_clk）作为输入，输出一路 50MHz 时钟（clk_50m），作为 DA 芯片的驱动时钟和 ROM IP 核的读时钟。

最后，由于本实验不需要模数转换（ADC），所以就不用添加 ILA 查看波形了。

综上所述，我们可以绘制出如下所示的系统框图：

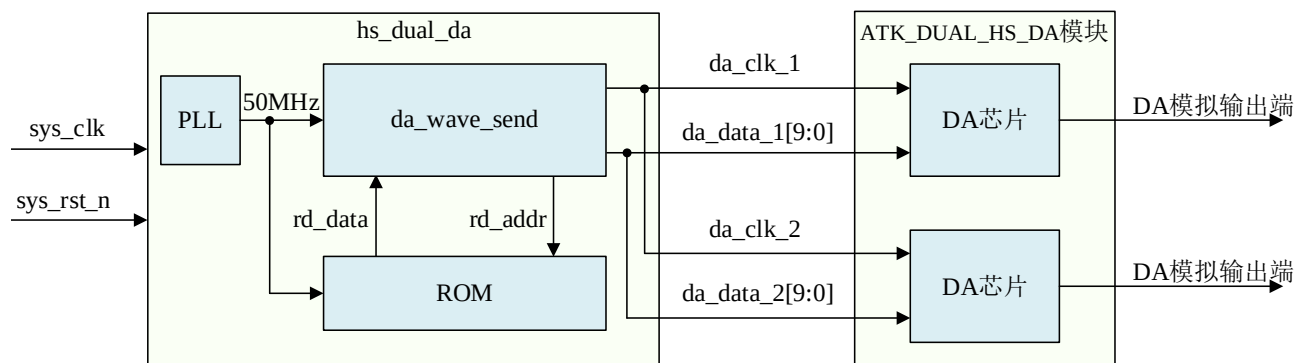


图 1.4.2 双路高速 DA 系统框图

由上图可知，本次设计有四个模块，分别为：顶层模块，用于例化和关联各个子模块之间的信号传递；PLL IP 核模块，用于产生实验所需的时钟信号；ROM IP 核模块，用于存储 DA 波形数据发送模块所需的波形数据；DA 波形数据发送模块，用于实现主控芯片与 DA 芯片之间的数据交互。

顶层模块端口与功能描述如下表所示：

表 1.4.1 顶层模块端口与功能描述

信号名	位宽	方向	端口说明
sys_clk	1	输入	系统时钟，50MHz
sys_rst_n	1	输入	系统复位按键，低电平有效
da_clk_1	1	输出	第一路DAC驱动时钟
da_data_1	10	输出	发送的第一路10位DAC数据
da_clk_2	1	输出	第二路DAC驱动时钟
da_data_2	10	输出	发送的第二路10位DAC数据

1.4.2 生成 COE 文件

首先我们使用 WaveToMem 软件来生成一个 COE 文件，软件设置参数如下所示：

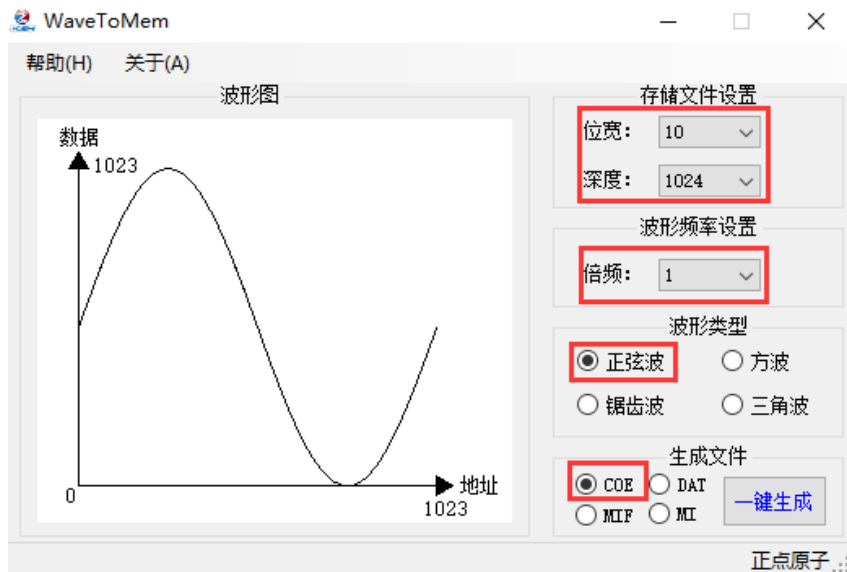


图 1.4.3 WaveToMem 软件设置

由于 ATK_DUAL_HS_DA 模块的 DA 芯片数据位宽为 10 位, 因此这里我们将位宽设置成 10 位。深度大家可以任意设置, 只是这里我们设置为了 1024 而已。只需要注意单个波形周期内的数据点越多, 最终输出的显示效果就越平滑。

1.4.3 ROM IP 核配置

工程中需要创建一个用于存储波形数据的单端口 ROM, 这里我们通过调用 BMG IP 核来实现。有关 BMG IP 核的内容, 还不熟悉的同学可以回顾下“IP 核之单端口 RAM 实验”和“RGB LCD 字符和图片显示实验”中的讲解, 这里我们仅对需要配置的参数进行展示说明。

“Basic”选项卡的配置如下图所示:

Component Name: rom_1024x10b

Basic | Port A Options | Other Options | Summary

Interface Type: Native (v) ☐ Generate address interface with 32 bits

Memory Type: Single Port ROM (v) ☐ Common Clock

ECC Options

ECC Type: No ECC (v)

☐ Error Injection Pins: Single Bit Error Injection (v)

Write Enable

☐ Byte Write Enable

Byte Size (bits): 9 (v)

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives.
Refer datasheet for more information.

Algorithm: Minimum Area (v)

Primitive: 8kx2 (v)

图 1.4.4 “Basic” 选项卡配置

这里我们将 IP 核命名为“rom_1024x10b”，其接口模式设置为“Native”、存储类型设置为“Single Port ROM”，即单端口 ROM。

“Port A Options”选项卡的配置如下图所示：

Basic | **Port A Options** | Other Options | Summary

Memory Size

Port A Width: 10 (Range: 1 to 4608 (bits))

Port A Depth: 1024 (Range: 2 to 1048576)

The Width and Depth values are used for Read Operation in Port A

Operating Mode: Write First (v) | Enable Port Type: Always Enabled (v)

Port A Optional Output Registers

☒ Primitives Output Register | ☐ Core Output Register

☐ SoftECC Input Register | ☐ REGCEA Pin

Port A Output Reset Options

☐ RSTA Pin (set/reset pin) | Output Reset Value (Hex): 0

☐ Reset Memory Latch | Reset Priority: CE (Latch or Register Enable) (v)

READ Address Change A

☐ Read Address Change A

图 1.4.5 “Port A Options” 选项卡配置

我们将 PortA 的位宽设置为 10，深度设置为 1024，以存储我们生成的 1024 个数据。此外，我们将使能引脚的类型设置为“Always Enabled”（即 ROM 一直处于使能的状态），然后取消 Port A Optional Output Register 的勾选。

接着我们来看下“Other Options”选项卡的配置，如下图所示：

The screenshot shows the 'Other Options' tab of an FPGA configuration interface. At the top, there are tabs for 'Basic', 'Port A Options', 'Other Options', and 'Summary'. Below the tabs, the 'Pipeline Stages within Mux' is set to 0, and 'Mux Size' is 1x1. The 'Memory Initialization' section contains a checked 'Load Init File' checkbox, a text field for 'Coe File' with the path 'e:/FPGA/hs_dual_da/doc/rom_1024x10b.coe', and 'Browse' and 'Edit' buttons. Below this is an unchecked 'Fill Remaining Memory Locations' checkbox and a 'Remaining Memory Locations (Hex)' field set to 0. The 'Structural/UniSim Simulation Model Options' section includes a description of warnings and a 'Collision Warnings' dropdown set to 'All'. The 'Behavioral Simulation Model Options' section at the bottom has two unchecked checkboxes: 'Disable Collision Warnings' and 'Disable Out of Range Warnings'.

图 1.4.6 “Other Options”选项卡配置

这里我们找到上一小节中生成的 COE 文件, 将其设置为 ROM 的初始化文件。最后我们直接点击“OK”按钮完成 BMG IP 核的配置。

至此 ROM IP 核的配置信息展示说明完成, 接着我们看下 PLL IP 核的配置。

1.4.4 PLL IP 核配置

本次实验我们需要使用 PLL IP 核来产生 50MHz 的 DAC 驱动时钟。有关 PLL IP 核的内容, 还不熟悉的同学可以回顾下“IP 核之 PLL 实验”中的讲解, 这里我们仅对需要配置的参数进行展示说明。

“Clocking Options”选项卡的配置如下图所示:

Component Name **clk_wiz_0**

Clocking Options | Output Clocks | Port Renaming | MMCM Settings | Summary

Clock Monitor

☐ Enable Clock Monitoring

Primitive

☒ MMCM ☐ PLL

Clocking Features | **Jitter Optimization**

☒ Frequency Synthesis ☐ Minimize Power ☒ Balanced

☒ Phase Alignment ☐ Spread Spectrum ☐ Minimize Output Jitter

☐ Dynamic Reconfig ☐ Dynamic Phase Shift ☐ Maximize Input Jitter filtering

☐ Safe Clock Startup

Dynamic Reconfig Interface Options

☒ AXI4Lite ☐ DRP ☐ Phase Duty Cycle Config ☐ Write DRP registers

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
	Primary	clk_in1	50.000	10.000 - 800.000	UI	0.010	Single ended clock capable pir
☐	Secondary	clk_in2	100.000	30.000 - 60.000		0.010	Single ended clock capable pi

图 1.4.7 “Clocking Options” 选项卡配置

这里我们可以保持默认的 IP 核命名, “Primitive” 选择使用 MMCM, 输入时钟频率改为板载晶振的频率 50MHz。

“Output Clocks” 选项卡的配置如下图所示:

Clocking Options		Output Clocks		Port Renaming	MMCM Settings		Summary				
		Requested	Actual		Requested	Actual	Requested	Actual			
☒	clk_out1	clk_out1	50.000	⊗	50.000000	0.000	⊗	0.000	50.000	50.0	BUFG
☐	clk_out2	clk_out2	100.000		N/A	0.000	N/A	50.000	N/A	BUFG	
☐	clk_out3	clk_out3	100.000		N/A	0.000	N/A	50.000	N/A	BUFG	
☐	clk_out4	clk_out4	100.000		N/A	0.000	N/A	50.000	N/A	BUFG	
☐	clk_out5	clk_out5	100.000		N/A	0.000	N/A	50.000	N/A	BUFG	
☐	clk_out6	clk_out6	100.000		N/A	0.000	N/A	50.000	N/A	BUFG	
☐	clk_out7	clk_out7	100.000		N/A	0.000	N/A	50.000	N/A	BUFG	

☐ USE CLOCK SEQUENCING

Clocking Feedback

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Source

☒ Automatic Control On-Chip
☐ Automatic Control Off-Chip
☐ User-Controlled On-Chip
☐ User-Controlled Off-Chip

Signaling

☒ Single-ended
☐ Differential

Enable Optional Inputs / Outputs for MMCM/PLL

☐ reset
☒ locked
☐ power_down
☐ input_clk_stopped
☐ clkfbstopped

Reset Type

☒ Active High
☐ Active Low

图 1.4.8 “Output Clocks” 选项卡配置

这里我们勾选一路时钟输出，输出频率为 50MHz。另外，我们取消 PLL 复位信号（reset）的勾选，其余选项保持不变，然后直接点击“OK”按钮完成 PLL IP 核的配置。

1.4.5 DA 波形数据发送模块设计

因为我们将正弦波的波形数据存储在了 ROM 中，而 DA 波形数据发送模块（da_wave_send）需要输出波形数据，所以 da_wave_send 模块与 ROM IP 核之间是存在数据交互的，也就是 da_wave_send 模块需要给出 ROM 读地址才能将 ROM 中的数据读取出来。由此我们可以知道 da_wave_send 模块的输入信号除了系统时钟和系统复位外，还需要一个用于接收 ROM 读数据的 10 位寄存器，这里我们将其命名为 rd_data；输出信号则必须要有读 ROM 所需的读地址信号，这里我们将其命名为 rd_addr，因为我们设置的 ROM 深度为 1024，所以读地址的位宽可以设置为 10 位。

因为 da_wave_send 模块的主要作用是实现主控芯片与 DA 芯片之间的数据交互，而想要和 DA 芯片进行数据交互，就必须要有给予它驱动时钟，所以模块的输出信号要有用于驱动 DAC 的时钟信号，这里我们将其命名为 da_clk；又因为 DA 芯片是将主控芯片产生的数字信号转换为模拟信号，所以模块还需要输出一个 10 位的数字信号（da_data）。综上，我们可以绘制出如下所示的模块框图：

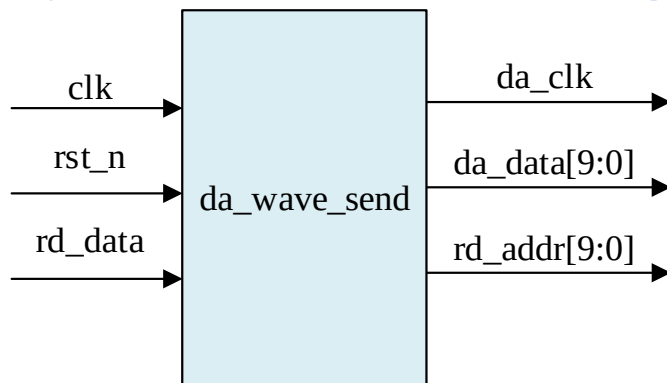


图 1.4.9 DA 波形数据发送模块框图

DA 波形数据发送模块端口与功能描述如下表所示:

表 1.4.2 DA 波形数据发送模块端口与功能描述

信号名	位宽	方向	端口说明
clk	1	输入	系统时钟, 50MHz
rst_n	1	输入	系统复位按键, 低电平有效
rd_data	10	输入	ROM中读出的数据
rd_addr	10	输出	读ROM地址
da_clk	1	输出	DAC驱动时钟
da_data	10	输出	输出给DAC的数据

绘制波形图

在编写代码前, 我们先梳理一下模块的端口时序。为了能够调节正弦波的输出频率, 我们引入了一个中间变量 `freq_cnt` (频率调节计数器), 我们可以通过改变该计数器的计数范围来改变读 ROM 的速度, 从而达到改变输出频率的目的。详细的说明我们稍后会在编写代码中进行讲解。

因为 ROM 中存储的是一个周期 (片段) 的正弦波数据, 所以为了 DA 能够输出连续的正弦波信号, 我们需要循环读取 ROM 中的所有数据, 即 ROM 读地址需要在 0~1023 之间循环计数。

还有一个需要注意的点是, 数据 `rd_data` 是在 `clk` (系统时钟) 的上升沿更新的, 而数据在中间位置 (即 `clk` 的下降沿) 时才是相对稳定的, 所以 DA 芯片要在 `clk` 的下降沿锁存数据才能保证数据的准确性, 但是 DA 实际上是在 `da_clk` (DAC 驱动时钟) 的上升沿锁存数据的, 因此 `da_clk` 与 `clk` 的相位需要相反。

综上, 我们可以绘制出如下所示的波形图:

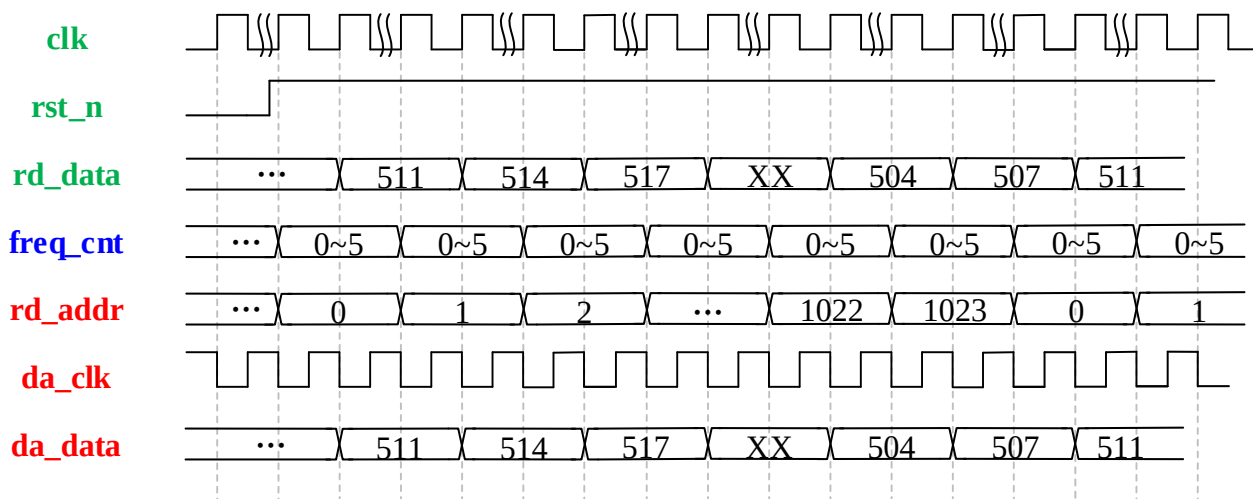


图 1.4.10 DA 波形数据发送模块端口波形图

由上图可知,我们将 `freq_cnt` 设置为在 0~5 之间进行循环计数,每当计数到 5 时,ROM 读地址(`rd_addr`)就加 1,当 `rd_addr` 计数到 1023 时,就会被清零并开始新一轮的计数。

编写代码

通过波形图的讲解,我们已经知道了 DA 波形数据发送模块(`da_wave_send`)的输入输出信号有哪些,也了解了内部信号是如何运行并产生模块所需的输出信号的,由此我们可以编写出如下所示的模块代码:

```

1 module da_wave_send(
2     input          clk          , //时钟
3     input          rst_n       , //复位信号, 低电平有效
4
5     input          [9:0]       rd_data    , //ROM 读出的数据
6     output reg     [9:0]       rd_addr    , //读 ROM 地址
7     //DA 芯片接口
8     output         da_clk      , //DAC 驱动时钟
9     output         [9:0]       da_data    //输出给 DA 的数据
10 );
11
12 //parameter
13 //频率调节控制
14 parameter FREQ_ADJ = 8'd5; //频率调节, FREQ_ADJ 的越大, 最终输出的频率越低, 范围 0~255
15
16 //reg define
17 reg [7:0] freq_cnt ; //频率调节计数器
18
19 //*****
20 /**                               main code
21 //*****
22
23 //数据 rd_data 是在 clk 的上升沿更新的, 所以 DA 芯片在 clk 的下降沿锁存数据是稳定的时刻
24 //而 DA 实际上在 da_clk 的上升沿锁存数据, 所以时钟取反, 这样 clk 的下降沿相当于 da_clk 的上升沿

```

```

25 assign da_clk = ~clk;
26 assign da_data = rd_data;    //将读到的 ROM 数据赋值给 DA 数据端口
27
28 //频率调节计数器
29 always @(posedge clk or negedge rst_n) begin
30     if(rst_n == 1'b0)
31         freq_cnt <= 8'd0;
32     else if(freq_cnt == FREQ_ADJ)
33         freq_cnt <= 8'd0;
34     else
35         freq_cnt <= freq_cnt + 8'd1;
36 end
37
38 //读 ROM 地址
39 always @(posedge clk or negedge rst_n) begin
40     if(rst_n == 1'b0)
41         rd_addr <= 10'd0;
42     else begin
43         if((freq_cnt == FREQ_ADJ) && (rd_addr < 10'd1023))
44             rd_addr <= rd_addr + 10'd1;
45         else if((freq_cnt == FREQ_ADJ) && (rd_addr == 10'd1023))
46             rd_addr <= 10'd0;
47         else
48             rd_addr <= rd_addr;
49     end
50 end
51
52 endmodule

```

因为我们用 WaveToMem 软件输出的是一个深度为 1024，倍频系数为 1 正弦波数据，又因为我们使用的输入时钟为 50MHz，所以一个完整的正弦波周期长度为 $1024 \times 20\text{ns} = 20480\text{ns}$ 。当 FREQ_ADJ 的值为 0 时 ROM 的读取速度最快，即输出的正弦波的最大频率为 $1\text{s}/20480\text{ns} (1\text{s} = 1000000000\text{ns}) \approx 48.83\text{KHz}$ ；当我们把 FREQ_ADJ 的值设置为 5 时，一个完整的正弦波周期长度就为 $20480\text{ns} \times (5+1) = 122880\text{ns}$ ，此时的输出频率约为 8.14KHz。当然我们也可以在 WaveToMem 软件设置中增加倍频系数或者增加 DA 的驱动时钟来提高正弦波的输出频率，需要注意的是驱动时钟频率不能超过芯片的最大转换速率所对应的时钟频率。

仿真波形如下所示：

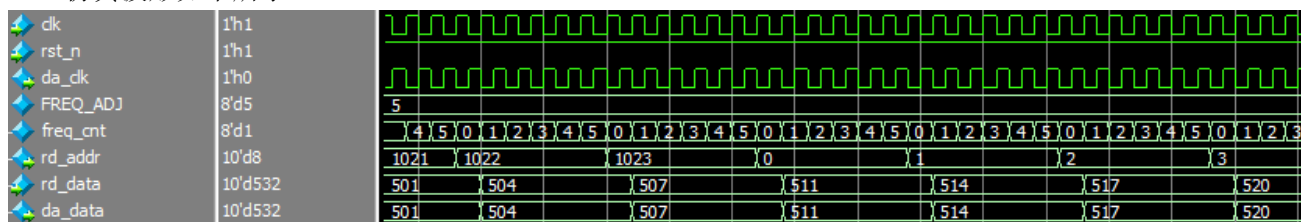


图 1.4.11 DA 波形数据发送模块仿真波形图

可以看出仿真结果与我们绘制的波形图是一致的，且从 ROM 中读出的数据也与我们生成的 COE 文件

中的数据是一致的, 由此可以说明我们的代码编写是没有问题。

1.4.6 顶层模块设计

实验工程的各功能模块均已讲解完毕, 接下来我们就需要根据总体模块设计中的讲解来将各个子模块例化到顶层模块中。

编写代码

顶层模块 (hs_dual_da) 的代码如下所示:

```

1 module hs_dual_da(
2     input          sys_clk    ,   //系统时钟
3     input          sys_rst_n  ,   //系统复位, 低电平有效
4     //DA 芯片接口
5     output         da_clk_1   ,   //第一路 DAC 驱动时钟
6     output [9:0]    da_data_1 ,   //第一路 DAC 数据
7     output         da_clk_2   ,   //第二路 DAC 驱动时钟
8     output [9:0]    da_data_2 ,   //第二路 DAC 数据
9 );
10
11 //wire define
12 wire [9:0] rd_addr ;           //ROM 地址
13 wire [9:0] rd_data ;          //ROM 数据
14 wire      clk_50m ;           //50MHz 时钟
15 wire      locked ;            //PLL 时钟锁定信号
16 wire      rst_n ;             //复位信号, 低有效
17
18 //*****
19 /**                               main code
20 //*****
21
22 //通过系统复位信号和 PLL 时钟锁定信号来产生一个新的复位信号
23 assign rst_n = sys_rst_n & locked;
24 assign da_clk_2 = da_clk_1;
25 assign da_data_2 = da_data_1;
26
27 //pll
28 clk_wiz_0 u_clk_wiz_0(
29     .clk_out1 (clk_50m),
30     .locked   (locked ),
31     .clk_in1  (sys_clk)
32 );
33
34 //ROM 模块
35 rom_1024x10b u_rom_1024x10b(

```

```
36     .addra      (rd_addr ),
37     .clka       (clk_50m ),
38     .douta      (rd_data )
39 );
40
41 //DA 数据发送
42 da_wave_send u_da_wave_send(
43     .clk         (clk_50m ),
44     .rst_n       (rst_n   ),
45     .rd_data     (rd_data ),
46     .rd_addr     (rd_addr ),
47     .da_clk      (da_clk_1 ),
48     .da_data     (da_data_1)
49 );
50
51 endmodule
```

在代码第 24 行和 25 行我们将两路 DAC 的驱动时钟线和数据线分别关联到了一起, 即 `da_clk_1` 和 `da_clk_2` 在同一条时钟线上, `da_data_1` 和 `da_data_2` 在同一条信号线上。因为这里我们使用的关联方法是将第一路 DAC 的驱动时钟和数据分别赋值给第二路 DAC 的驱动时钟和数据, 所以只需要将第一路 DAC 的驱动时钟和数据分别接到 `da_wave_send` 模块输出的 `da_clk` 和 `da_data` 信号线上即可实现两路 DAC 的正常工作, 如代码第 47 行和 48 行所示。当然, 关联的方法并不唯一, 大家也可以使用其它方法将两路信号关联在一起。

1.4.7 仿真验证

编写 TB 文件

我们接下来先对代码进行仿真, 本次实验的仿真主要是为了查看我们的代码是否正确读取出了 ROM 中存储的波形数据, 所以我们只需要编写时钟和复位这两个信号的激励即可, 其仿真代码如下所示:

```
1  `timescale 1ns / 1ps           //仿真单位/仿真精度
2
3  module tb_hs_dual_da();
4
5  //parameter define
6  parameter CLK_PERIOD = 20; //时钟周期 20ns
7
8  //reg define
9  reg      sys_clk;
10 reg      sys_rst_n;
11
12 //信号初始化
13 initial begin
14     sys_clk = 1'b0;
15     sys_rst_n = 1'b0;
```

```

16     #200
17     sys_rst_n = 1'b1;
18 end
19
20 //产生时钟
21 always #(CLK_PERIOD/2) sys_clk = ~sys_clk;
22
23 hs_dual_da u_hs_dual_da(
24     .sys_clk      (sys_clk      ),
25     .sys_rst_n    (sys_rst_n    ),
26     .da_clk_1     (),
27     .da_data_1    (),
28     .da_clk_2     (),
29     .da_data_2    ()
30 );
31
32 endmodule

```

仿真验证

接下来打开 Modelsim 软件对代码进行仿真，仿真的波形如下图所示：

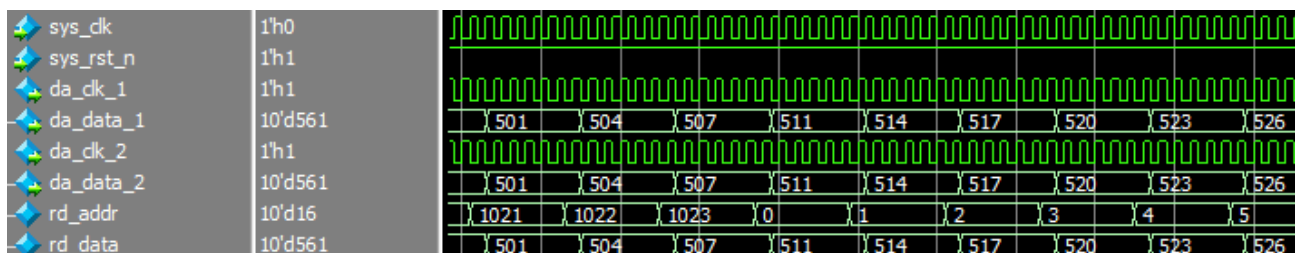


图 1.4.12 高速 DA 实验仿真波形图

由上图我们可以看出两路 DAC 的驱动时钟频率是一样的，DAC 数据是一致的，且 DAC 数据与我们生成的 COE 文件中的正弦波数据也是一致的，说明我们设计的代码能够通过双路高速 da 模块输出两路一模一样的正弦波信号。至此我们的仿真验证就结束了，可以进行最后的上板验证了。

1.5 下载验证

引脚约束

接下来我们对引脚进行分配，并上板验证。本次实验需要用到开发板 J2 扩展口来连接双路 DA 模块，本实验中的管脚分配如下表所示：

表 1.5.1 双路DA实验管脚分配

信号名	方向	管脚	端口说明	电平标准
sys_clk	input	R4	系统时钟，50Mhz	LVC MOS33
sys_rst_n	input	U2	系统复位，低有效	LVC MOS33
da_clk_1	output	E19	第一路DAC驱动时钟	LVC MOS33

da_data_1[0]	output	N22	输出给第一路DAC的数据	LVC MOS33
da_data_1[1]	output	M22	输出给第一路DAC的数据	LVC MOS33
da_data_1[2]	output	M21	输出给第一路DAC的数据	LVC MOS33
da_data_1[3]	output	L21	输出给第一路DAC的数据	LVC MOS33
da_data_1[4]	output	K22	输出给第一路DAC的数据	LVC MOS33
da_data_1[5]	output	K21	输出给第一路DAC的数据	LVC MOS33
da_data_1[6]	output	J22	输出给第一路DAC的数据	LVC MOS33
da_data_1[7]	output	H22	输出给第一路DAC的数据	LVC MOS33
da_data_1[8]	output	G22	输出给第一路DAC的数据	LVC MOS33
da_data_1[9]	output	G21	输出给第一路DAC的数据	LVC MOS33
da_clk_2	output	F21	第二路DAC驱动时钟	LVC MOS33
da_data_2[0]	output	N19	输出给第二路DAC的数据	LVC MOS33
da_data_2[1]	output	N18	输出给第二路DAC的数据	LVC MOS33
da_data_2[2]	output	M20	输出给第二路DAC的数据	LVC MOS33
da_data_2[3]	output	N20	输出给第二路DAC的数据	LVC MOS33
da_data_2[4]	output	J21	输出给第二路DAC的数据	LVC MOS33
da_data_2[5]	output	J20	输出给第二路DAC的数据	LVC MOS33
da_data_2[6]	output	G20	输出给第二路DAC的数据	LVC MOS33
da_data_2[7]	output	H20	输出给第二路DAC的数据	LVC MOS33
da_data_2[8]	output	L18	输出给第二路DAC的数据	LVC MOS33
da_data_2[9]	output	M18	输出给第二路DAC的数据	LVC MOS33

对应的 XDC 约束语句如下所示:

#时序约束

```
create_clock -period 20.000 -name sys_clk [get_ports sys_clk]
```

#IO 引脚约束

```
set_property -dict {PACKAGE_PIN R4 IOSTANDARD LVC MOS33} [get_ports sys_clk]
```

```
set_property -dict {PACKAGE_PIN U2 IOSTANDARD LVC MOS33} [get_ports sys_rst_n]
```

```
set_property -dict {PACKAGE_PIN G21 IOSTANDARD LVC MOS33} [get_ports {da_data_1[9]}]
```

```
set_property -dict {PACKAGE_PIN G22 IOSTANDARD LVC MOS33} [get_ports {da_data_1[8]}]
```

```
set_property -dict {PACKAGE_PIN H22 IOSTANDARD LVC MOS33} [get_ports {da_data_1[7]}]
```

```
set_property -dict {PACKAGE_PIN J22 IOSTANDARD LVC MOS33} [get_ports {da_data_1[6]}]
```

```
set_property -dict {PACKAGE_PIN K21 IOSTANDARD LVCMOS33} [get_ports {da_data_1[5]}]
set_property -dict {PACKAGE_PIN K22 IOSTANDARD LVCMOS33} [get_ports {da_data_1[4]}]
set_property -dict {PACKAGE_PIN L21 IOSTANDARD LVCMOS33} [get_ports {da_data_1[3]}]
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports {da_data_1[2]}]
set_property -dict {PACKAGE_PIN M22 IOSTANDARD LVCMOS33} [get_ports {da_data_1[1]}]
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports {da_data_1[0]}]
set_property -dict {PACKAGE_PIN E19 IOSTANDARD LVCMOS33} [get_ports da_clk_1]

set_property -dict {PACKAGE_PIN M18 IOSTANDARD LVCMOS33} [get_ports {da_data_2[9]}]
set_property -dict {PACKAGE_PIN L18 IOSTANDARD LVCMOS33} [get_ports {da_data_2[8]}]
set_property -dict {PACKAGE_PIN H20 IOSTANDARD LVCMOS33} [get_ports {da_data_2[7]}]
set_property -dict {PACKAGE_PIN G20 IOSTANDARD LVCMOS33} [get_ports {da_data_2[6]}]
set_property -dict {PACKAGE_PIN J20 IOSTANDARD LVCMOS33} [get_ports {da_data_2[5]}]
set_property -dict {PACKAGE_PIN J21 IOSTANDARD LVCMOS33} [get_ports {da_data_2[4]}]
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports {da_data_2[3]}]
set_property -dict {PACKAGE_PIN M20 IOSTANDARD LVCMOS33} [get_ports {da_data_2[2]}]
set_property -dict {PACKAGE_PIN N18 IOSTANDARD LVCMOS33} [get_ports {da_data_2[1]}]
set_property -dict {PACKAGE_PIN N19 IOSTANDARD LVCMOS33} [get_ports {da_data_2[0]}]
set_property -dict {PACKAGE_PIN F21 IOSTANDARD LVCMOS33} [get_ports da_clk_2]
```

上板验证

将双路高速 DA 模块插入开发板的 J2 扩展口, 连接时注意扩展口电源引脚方向和双路高速 DA 模块的电源引脚方向要一致, 如下图所示:

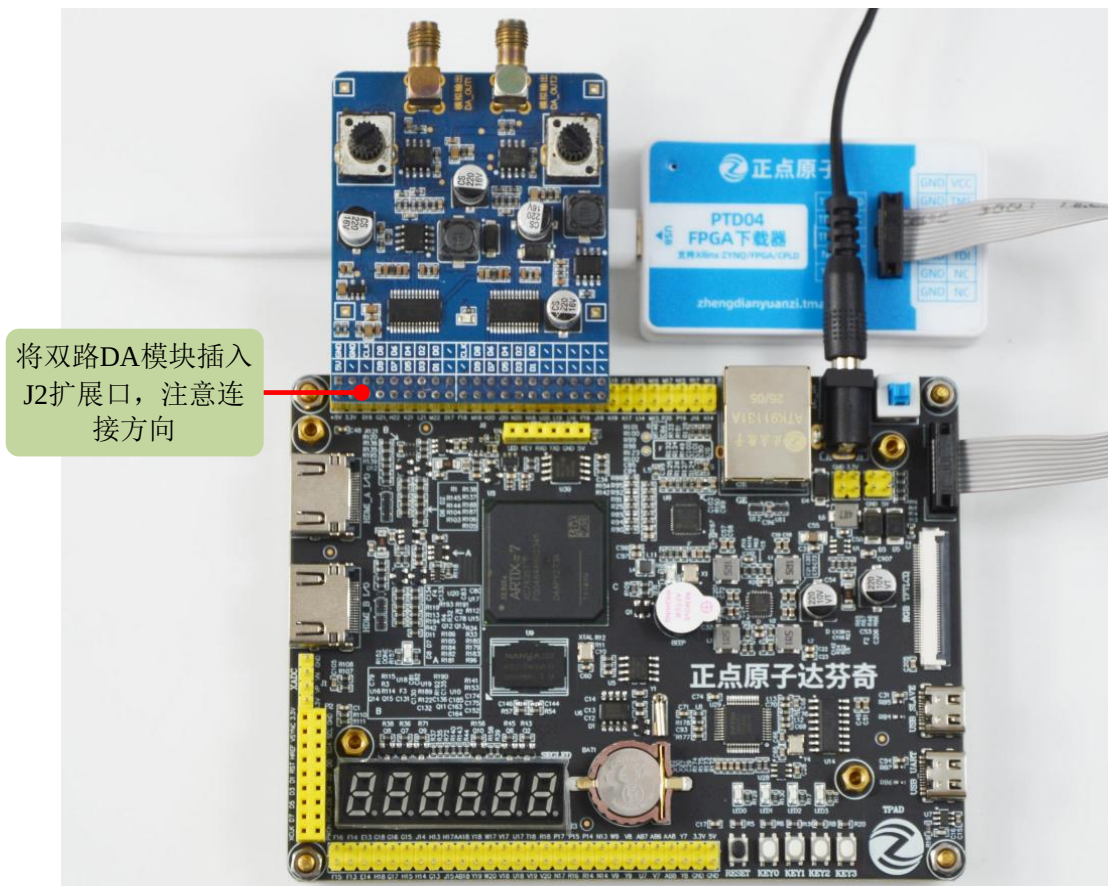


图 1.5.1 双路高速 DA 模块实物连接图

然后将下载器一端连接电脑, 另一端与开发板上的 JTAG 端口连接, 最后连接电源线并打开电源开关, 将工程生成的比特流文件下载到开发板中。

接着我们就可以使用示波器测量 DA 输出通道的波形了。首先将示波器带夹子的一端连接到开发板的 GND 上(可通过杜邦线连接至开发板上的任一 GND 管脚), 然后将另一端探针插入双路高速 DA 模块任一 DA 通道中间的金属圆圈内(注意将红色的保护套拿掉), 如下图所示:

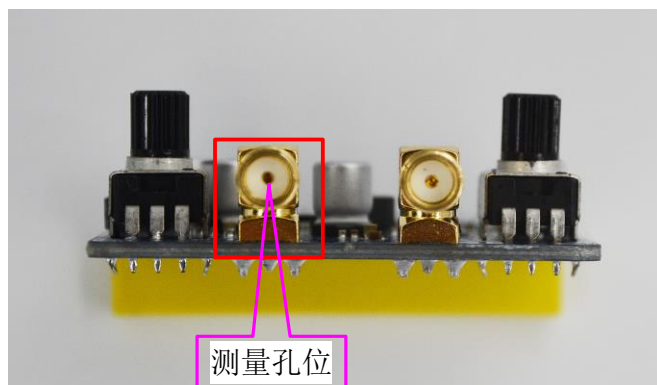


图 1.5.2 DA 模拟电压测量孔位

此时观察示波器可以看到模拟波的波形, 如果观察不到波形, 可查看示波器设置是否正确, 可以尝试按下示波器的“**AUTO**”, 再次观察示波器波形。示波器的显示界面如下所示:

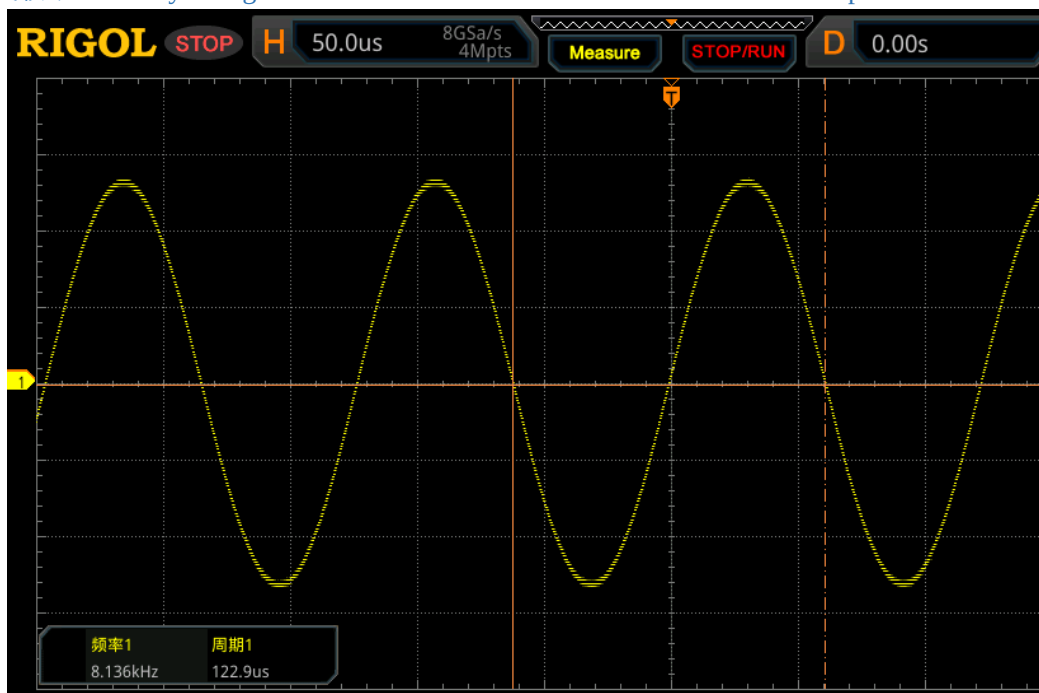


图 1.5.3 示波器显示界面

在两路 DAC 通道都能观察到正弦波波形, 且两路正弦波的频率 (8.14KHz 左右) 相同即可说明双路高速 DA 实验验证成功。

1.6 本章总结

经过本章的学习, 我们学会了如何实现开发板与双路高速 DA 模块之间的数据交互。且通过学习我们不难发现, 本次实验中的 DA 数据发送模块与“高速 AD/DA 实验”中的 DA 数据发送模块相比, 除了数据位宽不同外, 其余部分都是一模一样的。而数据位宽不同是因为两个模块所使用的 DAC 芯片的精度不同, 由此我们可以知道, 当我们使用其它精度的 DAC 芯片时, 如果驱动方式也是并行的话, 那么我们只需要改动程序设计中的数据位宽即可将其驱动起来。

1.7 拓展训练

本章我们使用双路高速 DA 模块输出了两路相同频率的正弦波信号, 如果大家还有余力的话不妨尝试着使用双路高速 DA 模块输出两路不同的波形信号。

这里为大家提供一个相对简单的设计思路: 因为要输出两种不同的波形, 所以我们需要对 `da_wave_send` 模块进行两次例化, 并将两路 DAC 相关的信号分别接入两个子模块中, 接着我们可以生成两个单端口 ROM 来分别存储两个波形数据文件 (有一定能力和基础的同学使用双端口 ROM 来实现当然也是可以的), 最后我们将两个 ROM 与两个 `da_wave_send` 模块分为两组, 并将组内的交互信号连接好即可实现输出两路不同波形信号的功能。大致的系统框图如下所示:

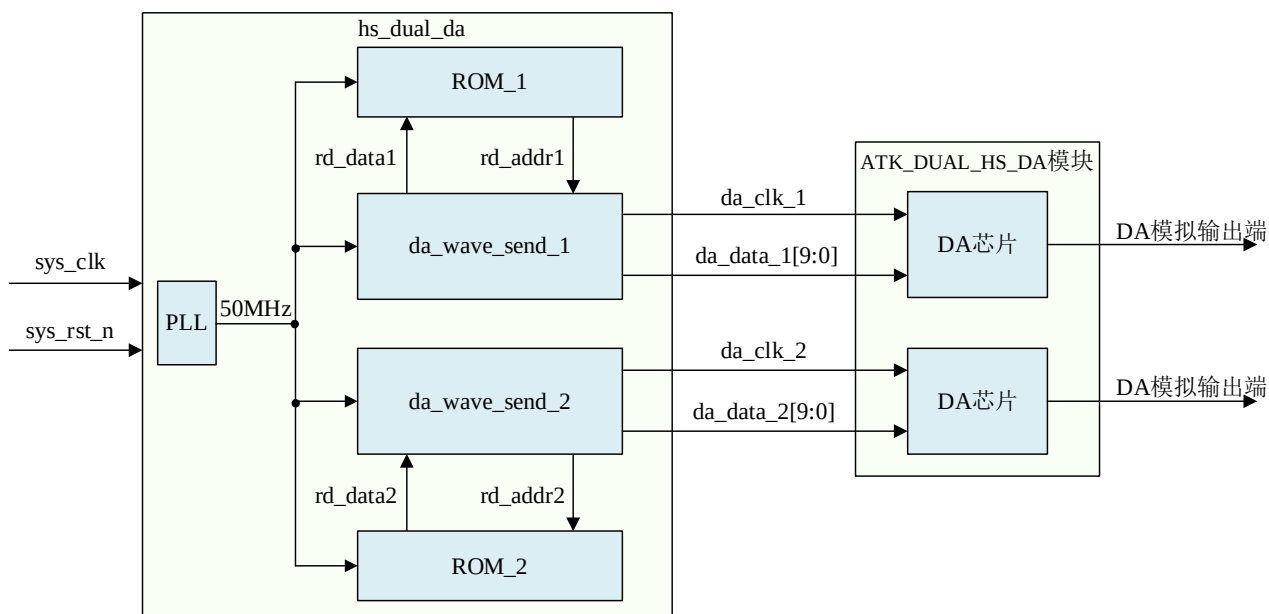


图 1.7.1 系统框图参考